Deliverable 2: Mid-Project Report

Group 5:
Bell, Aidan
Gebretsion, Samuel Million
Hanslod, Ahmed Shaukat
Mishra, Kanishk
Suganda, Emma Denise

Table of Contents

Table of Contents **0**
Level Design & Player Guidance **0**
- Story Elements 0
- Enemy Detection & Spatial Feedback 1

Narrative Design & World-Building **2**
- Narrative Context 2

Systems Design & Game Balance **4**
- Game Mechanics & Player Abilities 4
- Game Dynamics - Player Interaction & Feedback Loops 5
- Aesthetics & Sensory Feedback 5
- Computational Mechanics 5

Progression & Rewards **6**
- Progression System Audit 6
- Progression Playtesting 7

Testing & Iteration **8**
Technical Notes **9**
- Player Controller 9
- Guard Controller 9

Level Design & Player Guidance

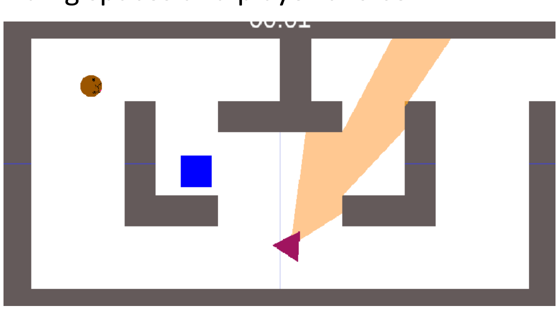
Story Elements

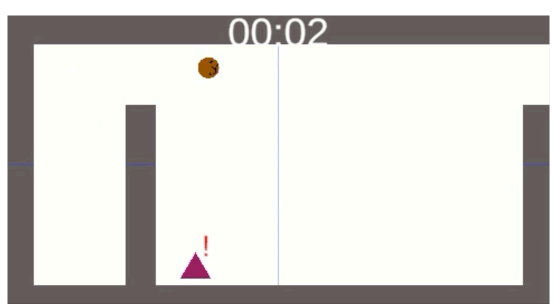
The game is themed around the two main characters, a dog and a cat, needing to escape an animal pound. The area is guarded by humans, and the characters will need to solve puzzles and fight enemies. The scope of the game is that of a Zelda dungeon—possibly smaller to fit the scope of the course—and the degree of freedom is limited in similar ways, where you need to unlock parts of the map to expand the size of your inhabitable world. So far we are more so just nailing the mechanics down for the game, and towards the latter couple weeks is where more of the narrative and aesthetic polish will come in.

Expanding the world will come from a mixture of puzzle solving, navigating, and mechanical execution, with the thrown in factor of remotely controlling both characters independently, changing with one press of a button. Certain aspects of the map will require work done by one character, to enable new paths and affordances for the other.

Enemy Detection & Spatial Feedback

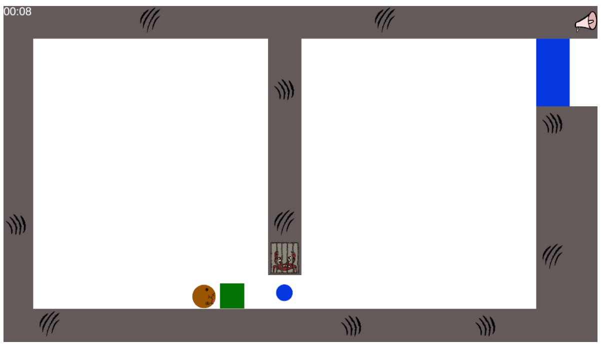
When a player is in the field of view of a guard, the guard will yell at the player and begin a chase, where they have to quickly run away. Guards are able to chase players using Unity's NavMesh system to track the player who is detected. A visual cue is given through an added exclamation mark around the guard to indicate that the guard detected the player. Additionally, players are given an audio cue indicating that a pursuit is about to begin. As a result, chases reinforce the need for effective level design that takes into account the need for hiding spaces and player choice.

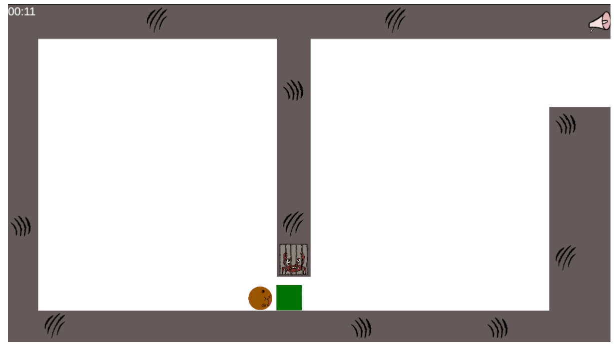




Player Interaction

One of the core interactive game mechanics are switches that need to be stood on or have a box on them to open a door, acting as a core spatial mechanic. Switches are both a puzzle element and directional cue for players. When a switch is activated, a simple cause and effect pattern is demonstrated through an audio cue and the door sliding open. This indicates the mechanics to the player without the need to walk them step by step through it. Using progression, the puzzles become more complex as players need to push boxes across guard patrolled areas. As such, players will gain a better understanding of spatial layout over time.





Visual Feedback

Furthermore, the updated sprite design and damage cosmetics for each animal now helps to better communicate the character and world state. As part of our previously planned improvements, level designs now include easily identifiable markers and in-game text, a means to provide guidance during puzzles and narrative context. Additionally, players can now switch between two playable characters. As a result, the design of puzzles requires players to think about how best to draw from different character abilities. Together, these visual feedback mechanics provide players the needed clarity to complete puzzles and a more intuitive gameplay that reinforces the need to escape the pound.

Narrative Design & World-Building

Narrative Context

The two main characters, a buff sphinx cat and a tiny chihuahua were just thrown into the pound styled like a human prison, whose prisoners are animals. The dog was thrown in jail for tax fraud, and the cat for beating up multiple veterinarian officers and eating the goldfish. They want to escape and return to the free world. They both either were captured at the same time, or one helps the other break out; either way a bond is formed and they are both in it to help each other escape.

Key figures who shape this world are the pound guards, who actively stop prisoners from escaping, and who put animals in the pound in the first place. More indirect key figures are humans who were the reason the two main characters are in the pound in the first place (abandonment, getting lost, getting snatched because of being strays, etc).

The environment will be very dirty and run down, which reflects how badly treated the animal prisoners are by the guards, enforcing why they should get out. Because the environment is run down, there will be a number of holes, gaps, and the like which the two characters can interact with to solve puzzles and progress the game.

The game will have a linear narrative structure that's told through prewritten in-game dialogue. The story is fixed with a single ending and Ludonarrative. Dialogue won't affect gameplay.

Environmental Storytelling

Much of the environmental storytelling will be left for the last few weeks where aesthetics and polish are the main focus. That said, we've jotted down our goals and ideas for what we'd want to implement

Sound effects:

1. Distant whimpers, and distressed animal noises
2. Creepy ambience
3. Water droplets
4. Chains
5. Footsteps
6. Cheap muffled speakers and alarms

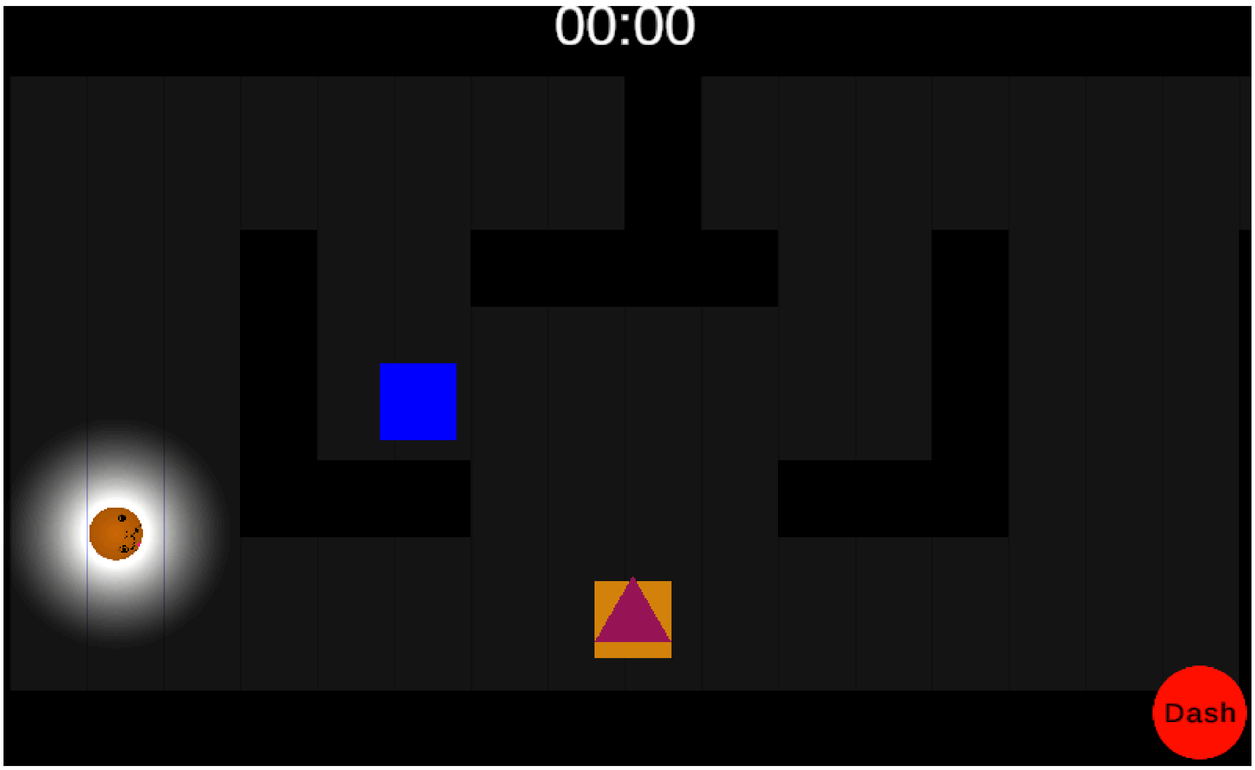
Props:

1. Broken cages/jail cells, catnip, guard tower (guards in general), security cameras, speakers/alarms

Lighting:

1. Flickering lights, spotlights, dark corners.

2. Spotlight can illuminate the character's radius as players travel through dark areas:



3. Graffiti and scratch marks on the wall

Systems Design & Game Balance

Game Mechanics & Player Abilities

The game features two playable characters with different mechanics. The cat is able to make a dash attack, which lets him overcome gaps in the terrain, move fast, and collide with objects or NPCs. The dog is able to grab and push objects around, such as moving a box to trap a guard, or moving onto a button that must be held down to activate. Both characters are able to make a simple hit attack which can be used to attack enemies, activate switches and buttons, or break objects. As there are two different characters who can be in different locations, there is also a camera mechanic. Pressing the space key switches the camera (and control of the character) to the other character. This allows players to strategically control both characters when separated by different puzzles, and expand their available options.

Through playtesting, we identified that the cat's dash mechanic was overpowered. The ability propelled the player too far, allowing them to easily escape guards, and essentially make them a non-factor. Also, without a cooldown period, there is only an animation delay that prevents the user from spamming the mechanic. As a solution, we reduced the dash distance and added a cooldown period to prevent players from overusing it. The adjustment is expected to make the game more challenging for players to escape enemies, while encouraging more strategic use of the cat's ability.

Game Dynamics - Player Interaction & Feedback Loops

In-game objects had no clear markers outside of color coding to indicate that the object is an interactable. To enhance the player interaction and feedback loop, we implemented visual cues like an outline around the object and distinct texture patterns. This change provides more clarity on what the player can actually interact with in the game, reducing the time players spend thinking about what they should be doing in a given moment.

Aesthetics & Sensory Feedback

During testing, we noticed that audio cues distracted the players too much and felt quite irritating. We addressed this issue by reducing the volume and switching to a more ambient sound track. The music now supports players during puzzle solving, rather than competing with gameplay. Another game balance issue had to do with the aesthetics of the guard. The color of the guards, dark navy blue, camouflaged quite well with the dark grey stone floors, making it difficult to notice guards. Guards are now a lighter shade of blue, providing a more intuitive and easy way to notice and locate an enemy within a room.

Computational Mechanics

A key computational mechanic within the game involves the guard AI, responsible for enforcing gameplay aspects such as stealth movement and pursuits. Upon testing, we observed that enemy guards were too persistent. Once a player was detected, a chase would occur that would last indefinitely, even through obstacle obstruction. This behavior distracts players from being able to solve the puzzle. To address this, once an enemy guard loses sight of the player, they will automatically abandon their pursuit and switch to patrolling their area. This change balanced the fairness and challenge of defeating enemy guards, while providing players the ability to recalculate their escape strategy in this dynamic world.

Progression & Rewards

Progression System Audit

As this game is a top-down puzzle adventure, world progression plays a big role in it. The game's world is one large 'dungeon' (the pound) made up of smaller rooms. Each of these small rooms has a puzzle and/or enemies. To win the game, the player has to explore all the rooms that'll lead them to escape the pound. To explore all the rooms, the player will have to solve the puzzle in one room in order to unlock another room. This creates a meaningful sense of progression through completing puzzles of increasing difficulty, as it builds up to the overall objective of escaping.

There are no unlockable abilities. The game relies on the characters' simple mechanics being used in different contexts to solve puzzles. There are also no unlockable items, as the reward of the game is tied to the amount of area the player can access. Though it isn't a perfect system, and is likely more reliant on player motivations than is ideal, it's a fair compromise for the scope of this class.

Additionally, rather than giving players more power directly through new abilities, players are given power by indirectly familiarising them with the game's mechanics. This is first done in simpler puzzles that introduce the game's mechanics, which will make more complex puzzles easier to handle later on. This does impose a difficulty on managing the game's difficulty curve. It may be hard to control on large maps, as player actions can lead to discovering high difficulty areas early on. However, this can be overcome and/or minimised in designing the level's layout and playtesting.

Weak Points:

- Limited rewards: Perhaps give players specific abilities after completing objectives or limited use power ups.
- Difficulty spikes: Depending on player actions, more careful control over mapping will guide players with a proper difficulty curve.

- Hard resets: Getting caught effectively restarts playthrough, stalls feeling of proper progression

Progression Playtesting

When playtesting the game with a focus on the progression systems a number of issues with our implementation were revealed. The most common thing noted about our progression system was that not many upgrades and unlocks were available. Most of the player mechanics were available from the start although this was not an issue from our perspectives. This was to prevent the players who are replaying the game from having to unlock the mechanics and allow them to breeze past the easier puzzles. A sense of progression was provided by puzzle mechanics being introduced one at a time giving a sense of upgrades to the complexity of puzzles.

A more serious issue we found was how repeatedly failing was very frustrating for the players. They felt stuck and frustrated when caught by guards on account of them being extraordinarily persistent. It was very easy to be caught unless the surrounding and player abilities were properly utilized. This frustration was mitigated in scenes testing specific mechanics due to them being much smaller and by extension less punishing. This resulted in the idea of checkpoints being suggested to minimize the frustration of playing as well as a timer that made the guard give up on the chase.

There was also positive feedback given for some parts of our progression systems. Players reported completing puzzles gives a feeling of satisfaction when appropriately challenging. Players reported a feeling of growth that came specifically with an increase in skill with the character mechanics. When the challenge of puzzles and evading guards matched with player skill, the players were the happiest with the game. This also meant the least satisfying aspect was the completion or discovery of simple puzzles that took little or no effort to go past.

Pain Points:

- High penalty for full play through (mitigated by test scenes effectively having checkpoints)
- Limited upgrades to player options as mechanics are available from the start.

Strengths:

- Puzzles and challenging interactions that matched with player skill felt highly rewarding.

Testing & Iteration

The playtesting process was a simple method of playing the game and reporting opinions on specific elements. Focus was drawn on specific mechanics by having the testing conducted on test unity scenes where those mechanics were being developed. A number of changes were proposed on a variety of aspects, many of which have been outlined in their respective sections, the following is the total summation of the main issues.

- **Issue:** The cat dash mechanic is overpowered as it propels the player too far, allowing them to easily escape enemies. Also, without a cooldown period, there is only an animation delay that prevents the user from spamming the mechanic.
  - **Proposed Adjustment:** Reduce the dash distance and add a cooldown period to prevent players from overusing it.
  - **Expected Impact:** The adjustment will make it more challenging for players to escape enemies and will encourage strategic use.
- **Issue:** There are no clear markers(outside of colour coding) that indicate interactable game objects in the world.
  - **Proposed Adjustment:** Adding in-game prompts and/or a sheen-like texture onto interactable game objects.
  - **Expected Impact**: Much more clarity on what the player can actually interact with in the game, reducing the time players spend thinking “What am I supposed to be doing right now?”
- **Issue**: The security officer’s getup was dark navy blue muddled with the dark grey stone floors and blended in too well.
  - **Proposed Adjustmen**t: Make his get-up a lighter shade of blue.
  - **Expected Impact**: Players will be able to easily clock that there is an enemy in this room and where exactly.
- **Issue**: Placeholder music distracted the players too much and felt slightly irritating
  - **Proposed Adjustment**: Turn the music down a tad, and choose a more ambient track.
  - **Expected Impact**: players will not feel as jaded by the music, and it can act more as background fodder for the player while solving puzzles
- **Issue:** Guard AI is too persistent, and continues to chase until it catches the player, even when out of sight
  - **Proposed Adjustment**: Limit duration guard can chase after losing sight of player and return guard to their spawn point

- **Expected Impact**: Allows the player to hide and retry puzzles instead of having to run away endlessly or lose

- **Issue:** The rewards given for progressing through the game are too focused on exploration and puzzle completion
  - **Proposed Adjustment**: Introduce specific mechanics, collectables, or power-ups as the player progresses through the game.
  - **Expected Impact**: Allows the player to have a stronger sense of progression by gaining options over completion rather than having all the options available from the start.
- **Issue:** Hard resets were a small issue for the smaller test scenes but full length level it would be very punishing for repeated failure.
  - **Proposed Adjustment**: Introduce checkpoints where the player will respawn if caught to mitigate punishments
  - **Expected Impact**: The player is less scared of making small mistakes, and reduces tension in favour of more thoughtful puzzle/exploration gameplay.

Technical Notes

Player Controller

The player controller script supports a variety of in-game features related to player state management, object detection, and character control. The script integrates Unity’s new input mapping system for better control of player actions. User input is converted into movement by mapping keys to different in-game directions. The script reads WASD and arrow key inputs as X and Y values, which are then converted into a 2D vector, normalized, and used to transform the player's position appropriately, allowing for a consistent and smooth control of player movement. Additionally, the script uses state machines to track player actions like normal movement and pushing. This allows for a seamless transition between states based on the player's input and or environment. For object interaction, the script uses raycasts to check if there is a pushable object in front of the player. Players can then press space when in range of a box to push it in a certain direction. Player controller settings such as movement speed and ray distance can be effortlessly updated in Unity’s inspector panel through the use of serializable fields.

Guard Controller

The guard controller script is responsible for managing the AI behavior of in-game enemy guards, allowing for patrols and chases. The script uses Unity’s NavMesh system to allow guards to navigate the world and find their way around obstacles. When a player is not detected, the guard will routinely patrol a predefined waypoint using Unity’s NavMeshAgent component. However, once a player is detected, a chase begins and Unity’s Raycast system is used to find a line of sight to the player, even through obstacle obstruction. Every tick, the script ensures that the direction of the guard matches the player's movement direction. When a line of sight to the player is lost, the guard automatically transitions into the patrol state. Furthermore, the script uses cone vision, multiple raycasts to form a cone shape, to create a realistic field of view for guards to spot players. As guards move and rotate, their vision updates in real-time, allowing for an accurate and intuitive way to spot players.